

PROJECT DOCUMENTATION

WhatNext Vision Motors

Driving Smarter Vehicle Sales and Dealer Operations with a Salesforce CRM Solution

Salesforce Developer Project

Team ID	SWTID-2026-4927
Team Leader	SATHISH S
Team Members	VIGNESH M YOGESWARAN C AUSTIN TINO ML MANOJKUMAR R
College / Organization	ALAGAPPA CHETTIAR GOVERNMENT COLLEGE OF ENGINEERING AND TECHNOLOGY
Department	CSE
Academic Year	2023-2027

WhatNext Vision Motors: Redefining the Vehicle Buying Experience Through Smart Automation

Project Summary

WhatNext Vision Motors is upgrading the way customers buy vehicles and the way its dealerships work by adopting a Salesforce CRM platform. The solution speeds up vehicle ordering by routing every order to the dealer closest to the customer and by refusing orders for vehicles that are not in stock. Automated flows keep order statuses current and send test-drive reminders by email without any manual follow-up.

The technical build combines Apex triggers for stock validation, a Batch Apex job for confirming pending orders, and a Scheduled Apex class that runs the batch every night. Together these features improve customer satisfaction, increase the accuracy of orders, and raise the overall efficiency of the dealer network.

1. INTRODUCTION

1.1 Project Overview

WhatNext Vision Motors adopted a Salesforce-based CRM to modernize how vehicles are ordered and how its dealerships are managed. The application captures customer orders, checks stock before an order is accepted, assigns the order to the most suitable dealer, schedules test-drive reminders and keeps order statuses up to date automatically.

Before this project, order details, stock figures and dealer information were spread across registers and spreadsheets that were never in sync. The new platform replaces these separate tools with one automated system that every user can access from a browser at any time.

1.2 Purpose

The purpose of the project is to bring vehicle ordering, stock control, dealer allocation and customer communication together on Salesforce. The system is designed to reduce manual effort, remove order errors, give real-time visibility of orders and stock, and deliver a smooth experience to the customer from the first enquiry to the final delivery.

1.3 Project Scope

- **In scope:** vehicle, dealer, customer, order, test-drive and service-request management; stock validation; dealer auto-assignment; test-drive reminder emails; nightly batch confirmation of pending orders; reports and dashboards.
- **Out of scope (for this release):** online payment, mobile application, live GPS-based dealer routing and integration with external ERP systems. These are listed under Future Scope.

1.4 Objectives

- Create a single, reliable store for vehicle, dealer and customer data.
- Stop orders for vehicles that have no stock left.
- Assign each new order to a dealer based on the customer's location without manual work.
- Remind customers about their scheduled test drives by email.
- Confirm pending orders automatically once stock becomes available.
- Provide managers with reports and dashboards for decision making.

1.5 Stakeholders

Stakeholder	Role in the system	Main benefit
Operations Manager	Monitors orders, stock and dealer performance	Single view of operations; fewer manual follow-ups
Dealer Executive	Handles orders assigned to the showroom	Receives only relevant orders
Customer	Places orders and books test drives	Clear availability and timely reminders
System Administrator	Configures and maintains the Salesforce org	Controlled, secure and scalable setup

2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

The table below captures the problems faced by the main users of the existing process, written in the "I am / I'm trying to / But / Because / Which makes me feel" format.

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Operations Manager	manage vehicle orders	stock information is maintained manually	systems are disconnected	stressed
PS-2	Customer	place a vehicle order	availability is unclear	updates arrive late	uncertain
PS-3	Dealer Executive	process incoming orders	I cannot tell which orders belong to my showroom	orders are not routed by location	confused

2.2 Empathy Map Canvas

User Persona: Operations Manager

Quadrant	Observation
Think & Feel	Worried about wrong orders, stock mismatches and unhappy customers
Hear	Complaints from dealers and customers about delays and cancellations
See	Paper registers, scattered spreadsheets and unread follow-up notes
Say & Do	Calls dealers repeatedly and updates stock sheets by hand
Pain	Delays, order cancellations and duplicated effort
Gain	Automation, instant stock checks and live dashboards

2.3 Brainstorming & Idea Prioritization

Ideas Considered

Idea	Level of automation	Remarks
Manual registers	Manual	Slow, error-prone, no reporting
Spreadsheet tracking	Semi-automated	Easy to start but cannot validate stock or assign dealers
Standalone web portal	Fully automated	High build and maintenance cost; no built-in CRM features
Salesforce CRM	Fully automated	Low-code automation, strong security, ready-made reporting

Final Selection

Salesforce CRM was chosen because it offers:

- Powerful automation through Flows, Apex and scheduled jobs
- Scalability for additional dealers and locations
- Real-time visibility through reports and dashboards

3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stages: Inquiry → Vehicle Selection → Order Placement → Dealer Assignment → Stock Validation → Test Drive → Confirmation → Reporting

Stage	Customer action	System response
Inquiry	Contacts the company about a vehicle	Customer record is created in Vehicle_Customer__c
Vehicle Selection	Chooses a model and variant	Vehicle details and availability are displayed
Order Placement	Submits the order	Vehicle_Order__c record is created with status Pending
Dealer Assignment	Waits for dealer contact	Flow assigns the dealer matching the customer location
Stock Validation	None	Trigger blocks the order if stock is zero
Test Drive	Books a test drive	Reminder email is sent one day before the drive
Confirmation	Receives confirmation	Order status moves to Confirmed; stock reduces by one
Reporting	None	Managers view reports and dashboards

3.2 Solution Requirements

Functional Requirements

FR No	Requirement	Description
FR-1	Vehicle management	Create and maintain vehicle models, price, stock and status
FR-2	Dealer management	Maintain authorized dealer details and locations
FR-3	Customer order creation	Capture customer orders with vehicle and date
FR-4	Stock validation	Reject orders when the vehicle is out of stock
FR-5	Dealer auto-assignment	Assign the dealer that matches the customer location
FR-6	Test drive scheduling	Book test drives and email a reminder before the date
FR-7	Reports & dashboards	Show order status, dealer performance and sales metrics

Non-Functional Requirements

NFR	Description
Usability	Lightning Experience UI that is easy to navigate
Security	Role-based access control and field-level security
Reliability	Automation that produces accurate and repeatable results
Performance	Fast order processing, with heavy work moved to batch jobs
Availability	24/7 access through the Salesforce cloud
Scalability	Support for multiple dealers and locations

3.3 Data Flow Diagram

Flow: Customer Input → Order Creation → Stock Validation → Dealer Assignment → Status Update → Reports

- **Customer Input:** customer and vehicle details are entered in Salesforce.
- **Order Creation:** a Vehicle_Order__c record is saved with status Pending.
- **Stock Validation:** the Apex trigger compares the order with Stock_Quantity__c.
- **Dealer Assignment:** the record-triggered flow finds the dealer for the customer's location.
- **Status Update:** the nightly batch job confirms pending orders when stock exists.
- **Reports:** dashboards read the order data and present live figures.

3.4 Technology Stack

Layer	Technology
User Interface	Salesforce Lightning Experience
Business Logic	Salesforce Flows (Record-Triggered Flows)
Database	Salesforce Custom Objects
Automation	Validation Rules, Apex Triggers
Processing	Batch Apex, Scheduled Apex
Reporting	Reports & Dashboards

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

The system removes fragmented operations by combining automated stock validation, dealer assignment and order processing inside one Salesforce org.

Problem	Solution in Salesforce CRM
Multiple scattered files	One centralized set of custom objects
Manual stock sheets	Stock_Quantity__c validated by an Apex trigger
Disconnected systems	A single Lightning app for all modules
Stressed manager, late updates	Automated flows, batch job and live dashboards

4.2 Proposed Solution

Parameter	Description
Problem	Manual and disconnected vehicle ordering
Solution	Salesforce CRM with automation
Innovation	Automatic dealer assignment based on customer location
Impact	Faster service and fewer order errors
Business Model	Authorized dealer network
Scalability	Multi-location, multi-dealer support

4.3 Solution Architecture

The solution is built entirely on the Salesforce platform. Custom objects hold the data, Flows and Apex handle the automation, Batch Apex handles bulk processing, and Reports and Dashboards present the results.

Component	Purpose
Vehicle Object	Stores model, price, stock and availability status
Dealer Object	Stores authorized dealer information
Customer Object	Stores customer contact details and preferences
Order Object	Tracks each vehicle purchase and its status
Test Drive Object	Tracks test-drive bookings
Service Request Object	Tracks servicing requests

Component	Purpose
Flows	Dealer assignment and test-drive reminder emails
Apex Triggers	Stock validation and stock reduction
Batch Jobs	Nightly confirmation of pending orders
Reports & Dashboards	Management insight into orders and performance

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

- Agile methodology with sprint-based delivery
- Work organised as Epics → User Stories → Story Points
- Six sprints of six working days each
- Velocity (average story points per sprint) used to estimate effort

Product Backlog, Sprint Schedule and Estimation

Sprint	Epic	USN	User Story / Task	Pts	Priority	Member
Sprint-1	Developer Setup	USN-1	As a system administrator, I want to create and configure a Salesforce developer account so that I can build and deploy the vehicle order system.	3	High	Member 1
Sprint-2	Data Modeling	USN-2	As an admin, I want to create the Vehicle, Dealer, Customer, Order, Test Drive and Service Request objects with relationships so that business data is stored systematically.	5	High	Member 2
Sprint-2	Data Modeling	USN-3	As a staff member, I want tabs and a Lightning App for all modules so that I can navigate vehicle orders and dealer operations easily.	5	High	Member 3
Sprint-3	Automation	USN-4	As a system, I want to validate stock, mandatory fields and order inputs so that invalid or incomplete orders are prevented.	3	High	Member 4
Sprint-3	Automation	USN-5	As a sales coordinator, I want flows to assign the nearest dealer and update order status so that processing is faster with minimal manual effort.	3	High	Member 4
Sprint-4	Apex	USN-6	As a system, I want Apex triggers to perform order confirmation and stock updates so that critical actions occur instantly and accurately.	5	High	Member 1
Sprint-4	Apex	USN-7	As an inventory manager, I want a batch job to confirm pending orders as stock becomes available so that inventory stays accurate.	5	High	Member 2
Sprint-5	Reports	USN-8	As a manager, I want reports on order status, dealer performance and sales so that I can monitor operations.	4	High	Member 3
Sprint-6	Dashboards	USN-9	As a manager, I want dashboards that visualise the reports so that I can make informed decisions at a glance.	4	Medium	Member 4

5.2 Project Tracker, Velocity & Burndown Chart

Sprint	Total Story Points	Duration	Start Date	Planned End Date	Points Completed	Release Date (Actual)
Sprint-1	3	6 Days	05 Oct 2026	10 Oct 2026	3	10 Oct 2026

Sprint	Total Story Points	Duration	Start Date	Planned End Date	Points Completed	Release Date (Actual)
Sprint-2	10	6 Days	12 Oct 2026	17 Oct 2026	10	17 Oct 2026
Sprint-3	6	6 Days	19 Oct 2026	24 Oct 2026	6	24 Oct 2026
Sprint-4	10	6 Days	26 Oct 2026	31 Oct 2026	10	31 Oct 2026
Sprint-5	4	6 Days	02 Nov 2026	07 Nov 2026	4	07 Nov 2026
Sprint-6	4	6 Days	09 Nov 2026	14 Nov 2026	4	14 Nov 2026

Dates and completed points above are sample values – replace them with your actual sprint dates before submission.

Velocity Calculation

Total story points = 3 + 10 + 6 + 10 + 4 + 4 = **37**. Number of sprints = 6. Average velocity = $37 \div 6 \approx 6.2$ story points per sprint.

Burndown Data

Sprint End	Points Remaining	Ideal Remaining
Start	37	37
Sprint-1	34	31
Sprint-2	24	25
Sprint-3	18	19
Sprint-4	8	12
Sprint-5	4	6
Sprint-6	0	0

6. PROJECT DEVELOPMENT PHASE

This phase explains, step by step, how the WhatNext Vision Motors application was built inside a Salesforce Developer Org. Every screen that should be captured as evidence is marked with a screenshot box.

What you will learn

- Data modelling with custom objects
- Creating fields and relationships
- Building a Lightning App with the App Manager
- Record-Triggered Flows
- Apex classes and Apex triggers
- Batch Apex
- Scheduled Apex
- Reports and dashboards

Milestone 1: Salesforce Developer Account

Activity 1: Creating a Developer Account

A free Developer Edition org gives a full copy of the Salesforce platform for building and testing the project.

Step 1: Open <https://developer.salesforce.com/signup> in your browser.

Step 2: Fill in the sign-up form with the following details:

- First name and Last name
- Email (a working email id is needed to receive the verification mail)
- Role: Developer
- Company: your college name
- Country: India
- Postal code: your area pin code
- Username: a unique id in e-mail format, for example `yourname@yourcollege.com` – it does not have to be a real mailbox

Step 3: Tick the agreement box and click **Sign me up**.

Activity 2: Account Activation

Step 1: Open the inbox of the email id you used. The verification email can take 5–10 minutes to arrive.

Step 2: Click **Verify Account** in the email.

Step 3: Set a new password, choose a security question, type the answer and click **Change Password**.

Step 4: You are redirected to the Salesforce Setup home page, which confirms that the org is ready.

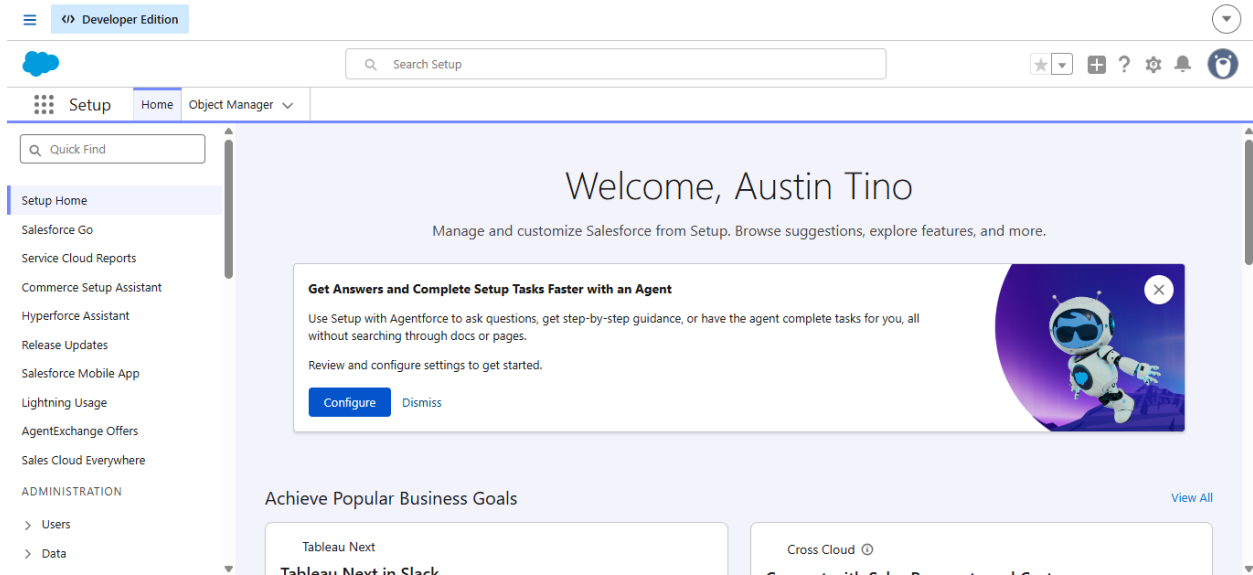


Figure 1: Salesforce Setup Home page

Milestone 2: Data Management – Objects

Objects are the tables of Salesforce. Six custom objects are required to store the business data of WhatNext Vision Motors.

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer and Orders
Vehicle_Dealer__c	Stores authorized dealer information	Related to Vehicles and Orders
Vehicle_Customer__c	Stores customer details	Related to Orders, Test Drives and Service Requests
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer, Vehicle and Dealer
Vehicle_Test_Drive__c	Tracks test-drive bookings	Related to Customer and Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer and Vehicle

To navigate to the Setup page

Click the **gear icon** at the top right of any Salesforce page and choose **Setup**.

Activity 1: Create the Vehicle Object

The Vehicle custom object keeps a clear record of every vehicle model, its price and its stock.

Step 1: From Setup, click **Object Manager** → **Create** → **Custom Object**.

Step 2: Enter Label = **Vehicle** and Plural Label = **Vehicles**.

Step 3: Under Record Name, enter Label = **Vehicle Name** and Data Type = **Text**.

Step 4: Under Optional Features tick **Allow Reports** and under Search Status tick **Allow Search**. Click **Save**.

The screenshot shows the Salesforce Setup interface in Developer Edition. The 'Object Manager' tab is selected, and the 'Vehicle' object is being configured. The left sidebar lists various configuration options, with 'Details' selected. The main area displays the following configuration details:

Field	Value
Description	
API Name	Vehicle_c
Custom	☒
Singular Label	Vehicle
Plural Label	Vehicles
Enable Reports	☒
Track Activities	
Track Field History	
Deployment Status	Deployed
Help Settings	Standard salesforce.com Help Window

Buttons for 'Edit' and 'Delete' are visible in the top right corner of the configuration area.

Figure 2: Vehicle object created

Activity 2: Create the Dealer Object

The Dealer object holds detailed information about every authorized dealer.

Step 1: Object Manager → **Create** → **Custom Object**.

Step 2: Label = **Vehicle Dealer**, Plural Label = **Vehicle Dealers**, Record Name = **Dealer Name** (Text).

Step 3: Tick **Allow Reports** and **Allow Search**, then click **Save**.

The screenshot shows the Salesforce Setup interface in Developer Edition. The 'Object Manager' tab is selected, and the 'Vehicle Dealer' object is being configured. The left sidebar lists various configuration options, with 'Details' selected. The main area displays the following configuration details:

Field	Value
Description	
API Name	Vehicle_Dealer__c
Custom	☒
Singular Label	Vehicle Dealer
Plural Label	Vehicle Dealers
Enable Reports	☒
Track Activities	
Track Field History	
Deployment Status	Deployed
Help Settings	Standard salesforce.com Help Window

Buttons for 'Edit' and 'Delete' are visible in the top right corner of the configuration area.

Figure 3: New Custom Object form – Vehicle Dealer

Activity 3: Create the Remaining Objects

Repeat the same procedure for the four remaining objects using the labels below.

Object Label	Plural Label	Record Name Label	Data Type
Vehicle Customer	Vehicle Customers	Customer Name	Text
Vehicle Order	Vehicle Orders	Order Number	Auto Number (VO-{0000})
Vehicle Test Drive	Vehicle Test Drives	Test Drive Number	Auto Number (TD-{0000})
Vehicle Service Request	Vehicle Service Requests	Request Number	Auto Number (SR-{0000})

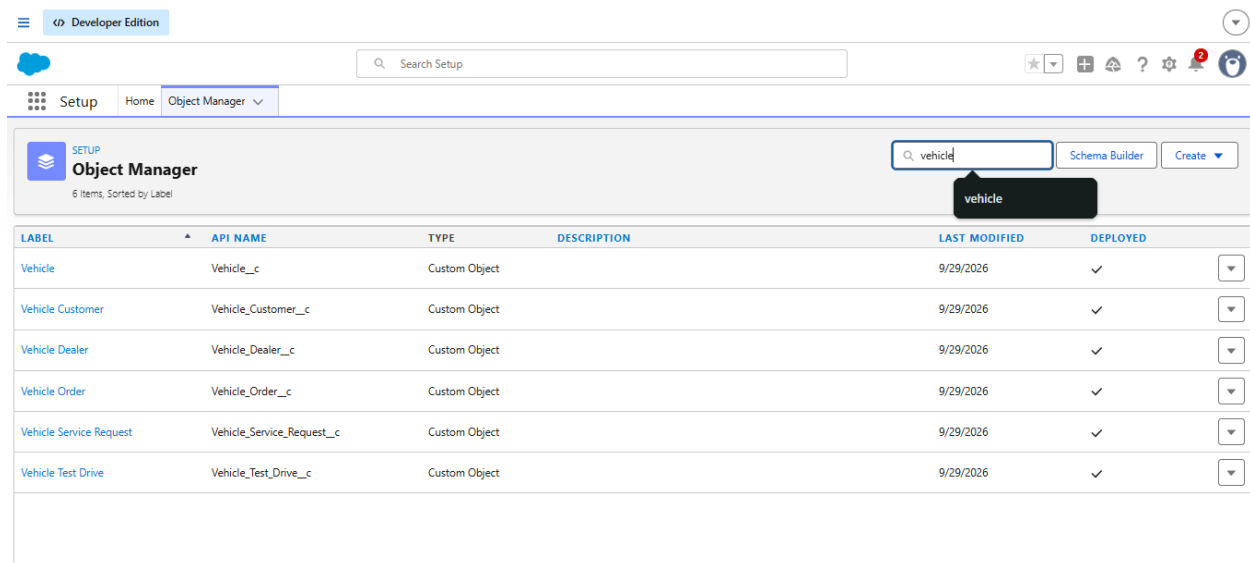


Figure 4: All six custom objects in Object Manager

Milestone 3: Data Management – Tabs

Tabs make each object reachable from the navigation bar so that users can open, create and search records quickly.

Activity 1: Creating a Custom Tab (Vehicle)

Step 1: In Setup, type **Tabs** in the Quick Find box and open **Tabs**. Under **Custom Object Tabs**, click **New**.

Step 2: Select Object = **Vehicle** and choose any Tab Style (for example, the car or truck icon). Click **Next**.

Step 3: Keep the default on **Add to Profiles** → **Next**, keep the default on **Add to Custom Apps** → **Next**, and click **Save**.

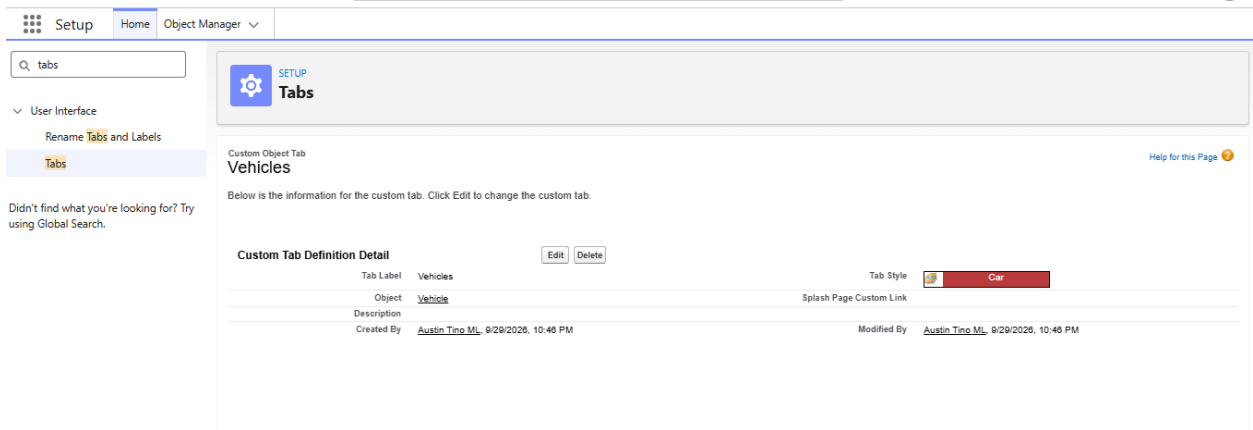


Figure 5: New Custom Object Tab – Step 1

Activity 2: Create Tabs for the Other Objects

Create custom tabs in the same way for Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive and Vehicle Service Request.

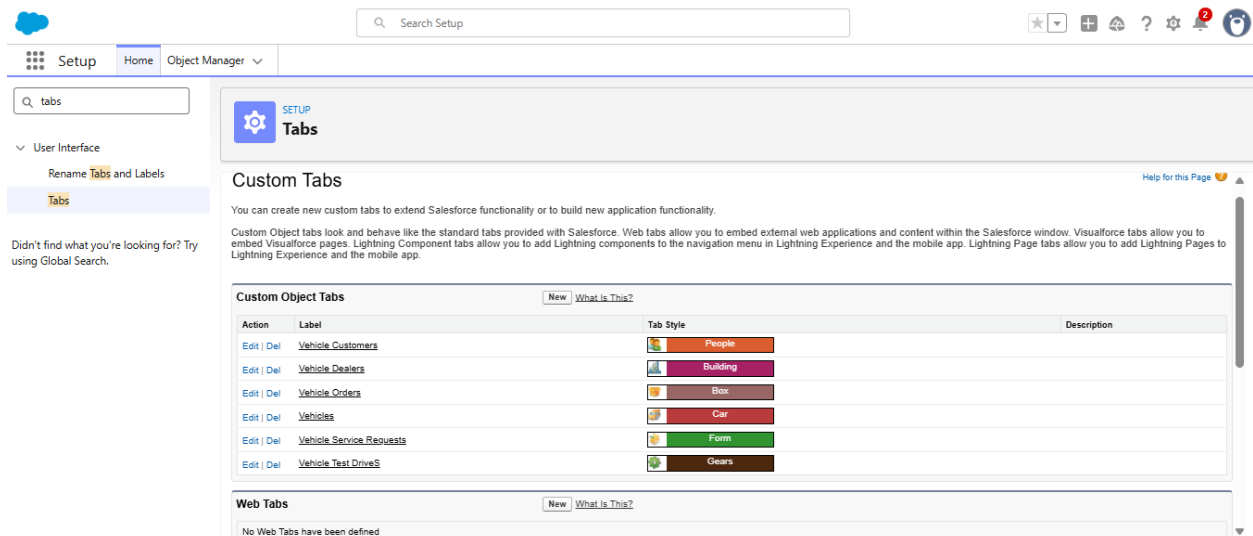


Figure 6: All custom object tabs created

Milestone 4: Data Management – App Manager

Activity 1: Create a Lightning App

A Lightning App groups the tabs and utilities needed by a team into one workspace. The app for this project is called **WhatNext Vision Motors**.

Step 1: In Setup, search for **App Manager** in Quick Find, open it and click **New Lightning App**.

Step 2: Enter the app details: App Name = **WhatNext Vision Motors**; Developer Name is filled automatically; add a meaningful description; the image is optional; keep the default primary colour.

Step 3: Click **Next** and keep the defaults on the App Options page, click **Next** and keep the defaults on the Utility Items page, then click **Next**.

Step 4: On Navigation Items, search for and move these items to the selected list: Vehicles, Vehicle Dealers, Vehicle Customers, Vehicle Orders, Vehicle Test Drives, Vehicle Service Requests, Reports and Dashboards. Make sure you select the **custom objects** created earlier. Click **Next**.

Step 5: On User Profiles, search for **System Administrator**, move it to Selected Profiles with the arrow button and click **Save & Finish**.

The screenshot shows the 'Lightning Experience App Manager' interface. On the left is a sidebar with navigation links like 'Setup Home', 'Salesforce Go', and 'Administration'. The main area displays a table of 32 items, sorted by App Name. The table has columns for App Name, Developer Name, Description, Last Modified Date, App Type, and Visibility. The 'WhatNext Vision Motors' app is listed at the bottom of the table.

App Name	Developer Name	Description	Last Modified Date	App Type	Visibl...
22 Sales	LightningSales	Manage your sales process with accounts, leads, opportunities, and more	9/21/2026, 10:51 AM	Lightning	✓
23 Sales Cloud Mobile	SalesCloudMobile	New seller focused mobile first experience	9/21/2026, 10:51 AM	Lightning	✓
24 Sales Console	LightningSalesConsole	(Lightning Experience) Lets sales reps work with multiple records on on...	9/21/2026, 10:51 AM	Lightning	✓
25 Salesforce Chatter	Chatter	The Salesforce Chatter social network, including profiles and feeds	9/21/2026, 10:51 AM	Classic	✓
26 Salesforce Scheduler Setup	LightningScheduler	Set up personalized appointment scheduling.	9/21/2026, 10:54 AM	Lightning	✓
27 Service	Service	Manage customer service with accounts, contacts, cases, and more	9/21/2026, 10:51 AM	Classic	✓
28 Service Console	LightningService	(Lightning Experience) Lets support agents work with multiple records a...	9/21/2026, 10:51 AM	Lightning	✓
29 Shield	Shield	Monitor your Salesforce apps and data. Discover, track, and encrypt sen...	9/21/2026, 10:51 AM	Lightning	✓
30 Site.com	Sites	Build pixel-perfect, data-rich websites using the drag-and-drop Site.co...	9/21/2026, 10:51 AM	Classic	✓
31 Subscription Management	RevenueCloudConsole	Get started automating your revenue processes	9/21/2026, 10:51 AM	Lightning	✓
32 WhatNext Vision Motors	WhatNext_Vision_Motors		9/29/2026, 10:50 PM	Lightning	✓

Figure 7: WhatNext Vision Motors app in the App Launcher

Milestone 5: Data Management – Fields & Relationships

Fields describe the data stored in each object. The tables below list the key fields to be created for every object. Record Name fields (Vehicle Name, Dealer Name, Customer Name and the auto-number names) already exist as the standard Name field.

Key Fields for Each Object

1. Vehicle__c

Field API Name	Data Type	Details
Vehicle_Model__c	Picklist	Sedan, SUV, EV, Hatchback
Stock_Quantity__c	Number (18,0)	Units currently in stock
Price__c	Currency	Ex-showroom price
Dealer__c	Lookup	Lookup to Vehicle_Dealer__c
Status__c	Picklist	Available, Out of Stock, Discontinued

2. Vehicle_Dealer__c

Field API Name	Data Type	Details
Dealer_Location__c	Text	City / area of the showroom
Dealer_Code__c	Auto Number	DLR-{0000}
Phone__c	Phone	Dealer contact number
Email__c	Email	Dealer email id

3. Vehicle_Order__c

Field API Name	Data Type	Details
Customer__c	Lookup	Lookup to Vehicle_Customer__c
Vehicle__c	Lookup	Lookup to Vehicle__c
Assigned_Dealer__c	Lookup	Lookup to Vehicle_Dealer__c (filled by the flow)
Order_Date__c	Date	Date the order was placed
Status__c	Picklist	Pending, Confirmed, Delivered, Canceled

4. Vehicle_Customer__c

Field API Name	Data Type	Details
Email__c	Email	Used for reminder emails
Phone__c	Phone	Customer contact number

Field API Name	Data Type	Details
Address__c	Text	City / area – matched with Dealer_Location__c
Preferred_Vehicle_Type__c	Picklist	Sedan, SUV, EV, Hatchback

5. Vehicle_Test_Drive__c

Field API Name	Data Type	Details
Customer__c	Lookup	Lookup to Vehicle_Customer__c
Vehicle__c	Lookup	Lookup to Vehicle__c
Test_Drive_Date__c	Date	Scheduled date of the drive
Status__c	Picklist	Scheduled, Completed, Canceled

6. Vehicle_Service_Request__c

Field API Name	Data Type	Details
Customer__c	Lookup	Lookup to Vehicle_Customer__c
Vehicle__c	Lookup	Lookup to Vehicle__c
Service_Date__c	Date	Requested service date
Issue_Description__c	Text Area	Description of the problem
Status__c	Picklist	Requested, In Progress, Completed

Object Field Creation – Activity 1 (Picklist field)

Step 1: Go to Setup → **Object Manager**, type **Vehicle** in Quick Find and click the Vehicle object.

Step 2: Click **Fields & Relationships** → **New**.

Step 3: Select the data type **Picklist** and click **Next**.

Step 4: Enter Field Label = **Vehicle Model**, type the picklist values (Sedan, SUV, EV, Hatchback – one per line), then click **Next** → **Next** → **Save & New**.

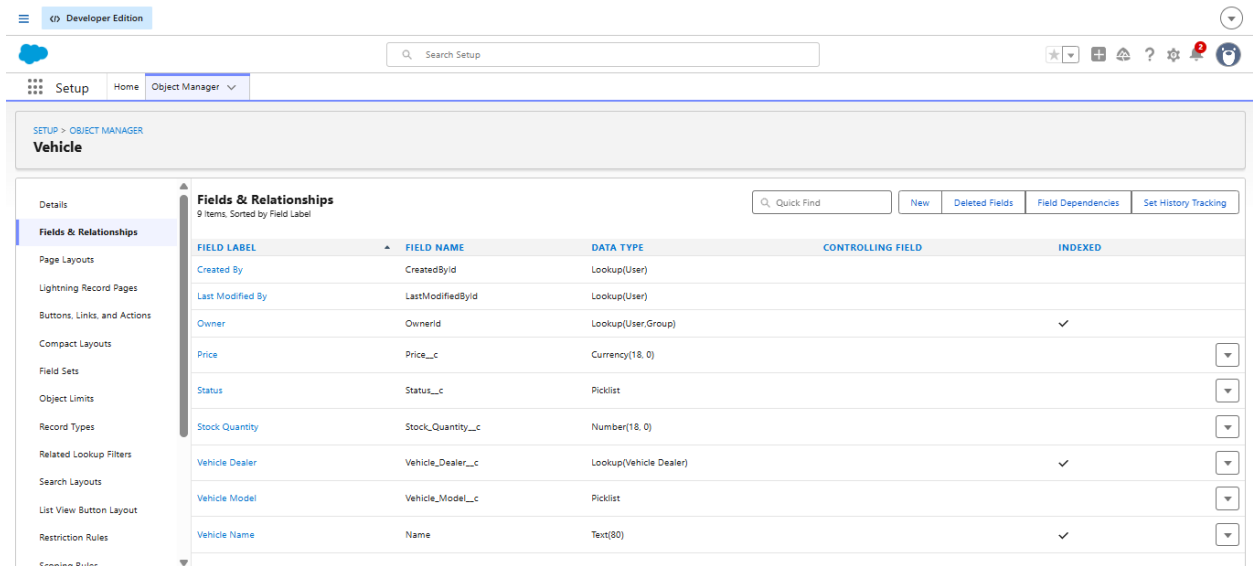


Figure 8: Picklist values entered

Object Field Creation – Activity 2 (Number field)

Step 1: Repeat Steps 1 and 2 above and select the data type **Number**, then click **Next**.

Step 2: Enter Field Label = **Stock Quantity**, Length = 18, Decimal Places = 0. Click **Next** → **Next** → **Save & New**.

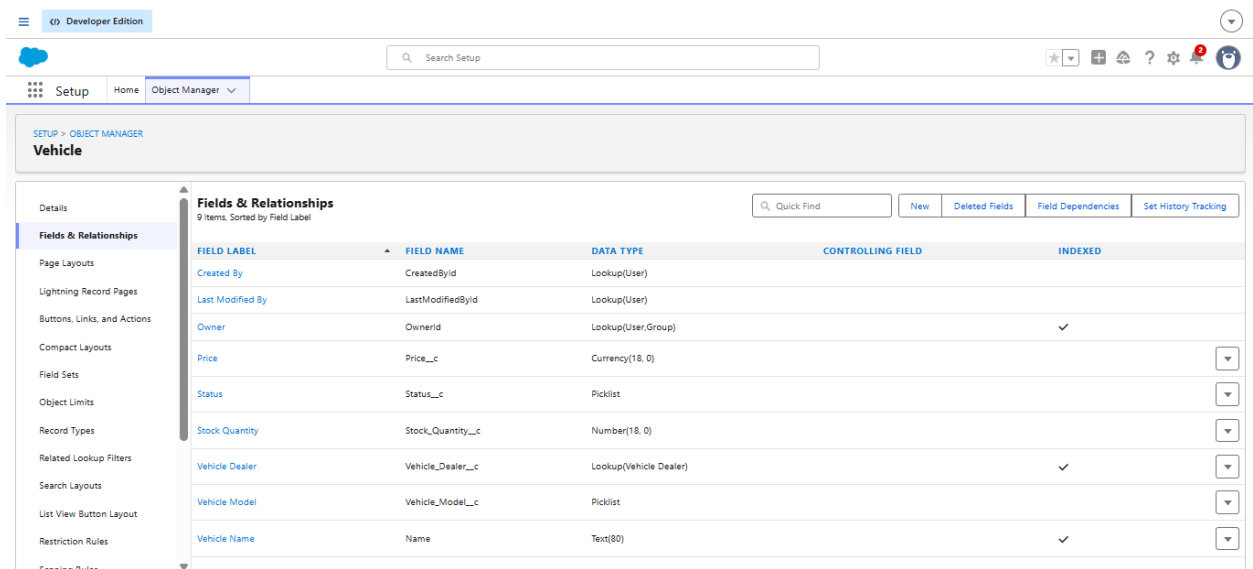


Figure 9: Number field – Stock Quantity

Relationship Field Creation – Lookup from Vehicle to Dealer

Step 1: Click **New** and choose the data type **Lookup Relationship**, then click **Next**.

Step 2: In **Related To**, choose **Vehicle Dealer** and click **Next**.

Step 3: Enter Field Label = **Dealer** (Field Name fills in automatically) and click **Next**.

Step 4: Keep field-level security as default (Visible) and click **Next** → **Next** → **Save**.

The screenshot shows the Salesforce Setup interface. The breadcrumb trail is 'SETUP > OBJECT MANAGER > Vehicle'. The left sidebar contains a menu with 'Fields & Relationships' selected. The main content area displays a table of fields for the 'Vehicle' object. The table has columns: FIELD LABEL, FIELD NAME, DATA TYPE, CONTROLLING FIELD, and INDEXED. There are 9 items listed, sorted by Field Label. The 'Vehicle Dealer' field is highlighted in blue.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Price	Price__c	Currency(18, 0)		
Status	Status__c	Picklist		
Stock Quantity	Stock_Quantity__c	Number(18, 0)		
Vehicle Dealer	Vehicle_Dealer__c	Lookup(Vehicle Dealer)		✓
Vehicle Model	Vehicle_Model__c	Picklist		
Vehicle Name	Name	Text(80)		✓

Figure 10: Choose the field type – Lookup Relationship

More Fields Creation

Create all remaining fields listed in the Key Fields tables for every object. For each object, take a screenshot of the finished Fields & Relationships list (keep the screenshots for Vehicle and Vehicle Order below; add the others in the same way if your guide requires them).

This screenshot is identical to Figure 10, showing the 'Fields & Relationships' list for the 'Vehicle' object in Salesforce Setup. The 'Vehicle Dealer' field is highlighted in blue.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Price	Price__c	Currency(18, 0)		
Status	Status__c	Picklist		
Stock Quantity	Stock_Quantity__c	Number(18, 0)		
Vehicle Dealer	Vehicle_Dealer__c	Lookup(Vehicle Dealer)		✓
Vehicle Model	Vehicle_Model__c	Picklist		
Vehicle Name	Name	Text(80)		✓

Figure 11: Vehicle – Fields & Relationships list

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Assigned Dealer	Assigned_Dealer__c	Lookup(Vehicle Dealer)		✓
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Order date	Order_date__c	Date		
Owner	OwnerId	Lookup(User,Group)		✓
Status	Status__c	Picklist		
Vehicle	Vehicle__c	Lookup(Vehicle)		✓
Vehicle Customer	Vehicle_Customer__c	Lookup(Vehicle Customer)		✓
Vehicle Order Number	Name	Auto Number		✓

Figure 12: Vehicle Order – Fields & Relationships list

Schema Builder (Data Model View)

Schema Builder displays the objects and the lookup relationships between them. Open **Setup** → **Schema Builder**, select the six custom objects and capture the diagram.

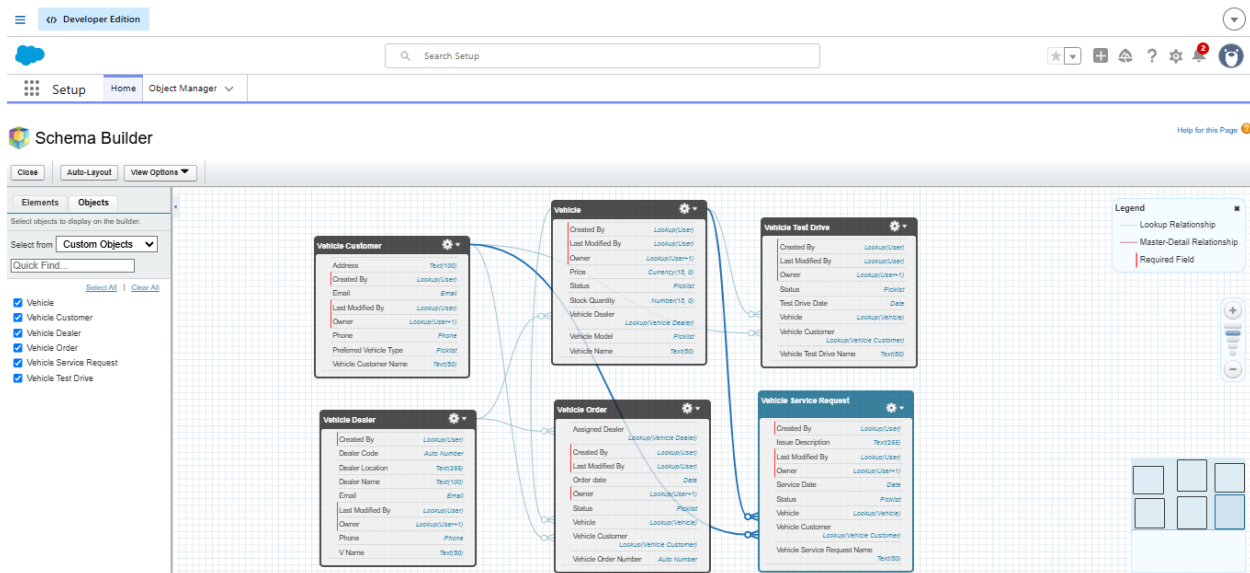


Figure 13: Schema Builder – data model

Milestone 6: Automation

Record-triggered flows run in the background whenever a record is created or updated. Two flows are built: one that assigns the dealer to a new order and one that e-mails a test-drive reminder.

Flow 1: Assign the Nearest Dealer to the Customer's Location

Step 1: In Quick Find type **Flows**, open **Flows** and click **New Flow**.

Step 2: Select **Start From Scratch** and click **Next**.

Step 3: Select **Record-Triggered Flow** and click **Create**.

Step 4: Configure the start element: Object = **Vehicle Order**; Trigger the flow when **A record is created**; Entry Conditions = **All Conditions Are Met (AND)** with Status__c Equals Pending; Optimize the flow for **Actions and Related Records**.

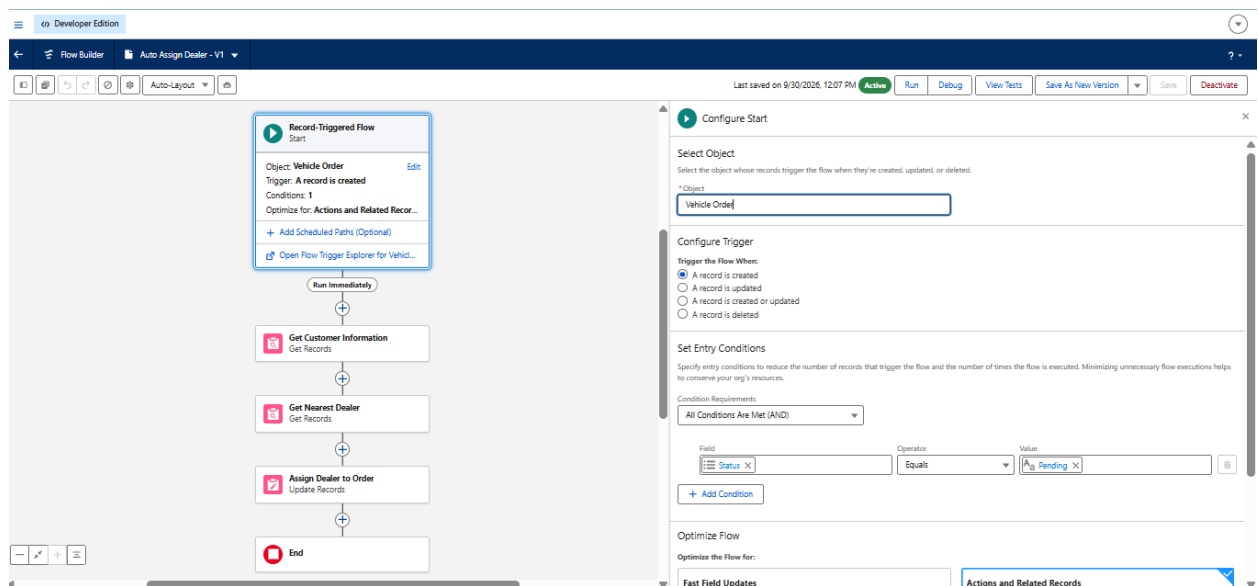


Figure 14: Configure Start – Vehicle Order

Step 5: Click the + icon and add a **Get Records** element. Label = **Get Customer Information**; Object = **Vehicle Customer**; Condition: Id Equals {!\$Record.Customer__c}; store only the first record with all fields.

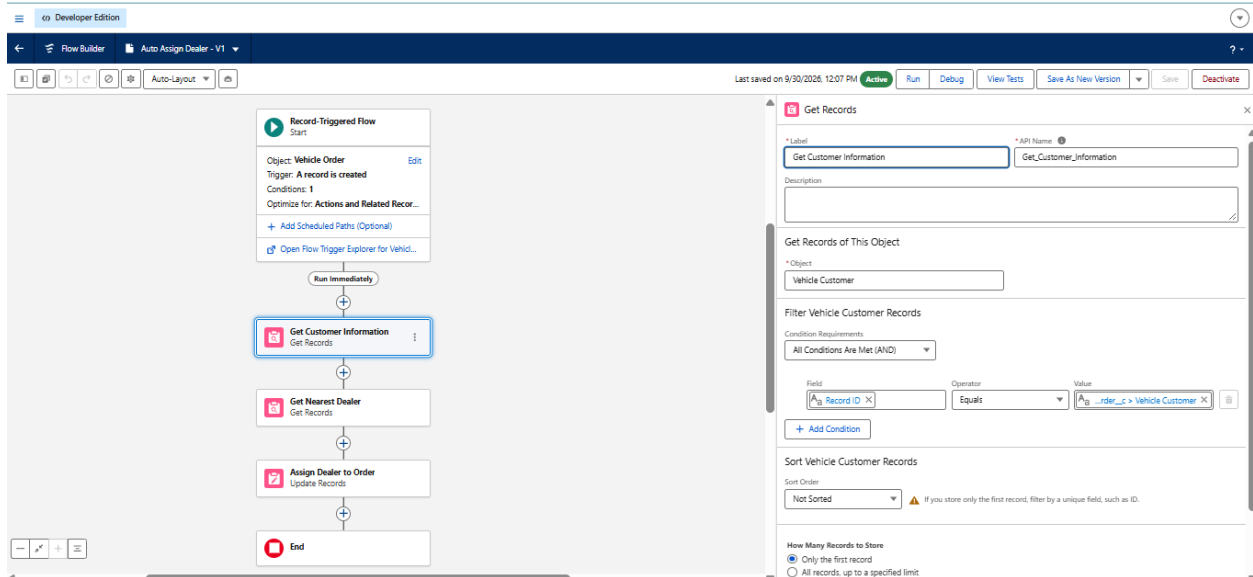


Figure 15: Get Records – Get Customer Information

Step 6: Add another **Get Records** element. Label = **Get Nearest Dealer**; Object = **Vehicle Dealer**; Condition: **Dealer_Location__c Equals {!Get_Customer_Information.Address__c}**; store only the first record.

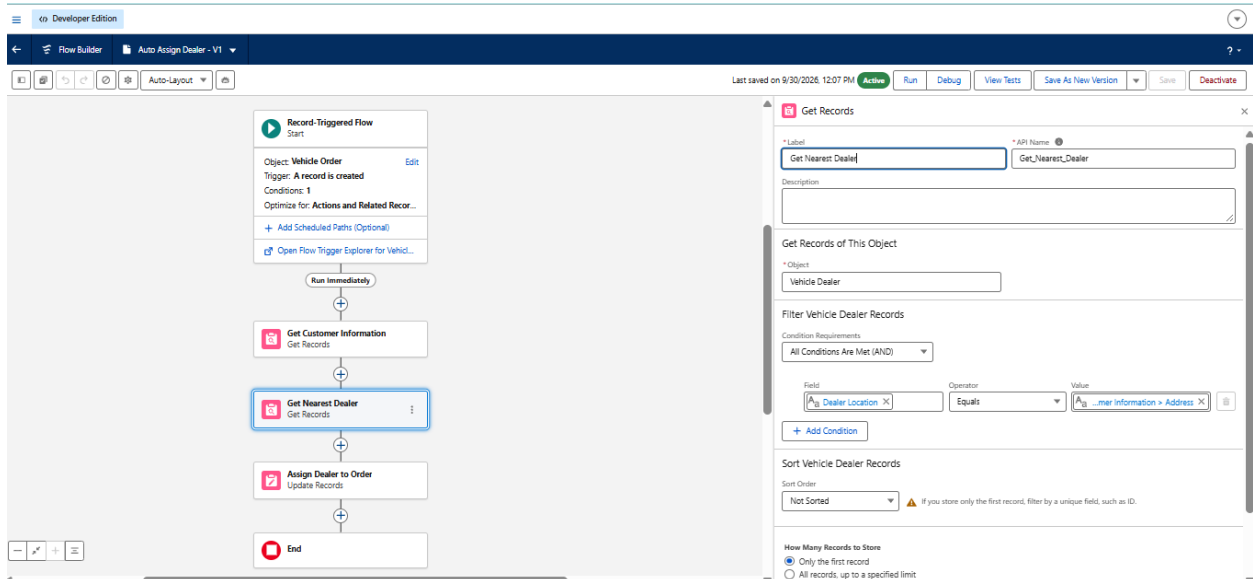


Figure 16: Get Records – Get Nearest Dealer

Step 7: Add a **Decision** element named **Dealer Found?** with one outcome, **Yes**, where **{!Get_Nearest_Dealer} Is Null** is **False**. This prevents an error when no dealer matches the customer location.

Step 8: On the **Yes** path add an **Update Records** element. Label = **Assign Dealer to Order**; choose **Use the vehicle order record that triggered the flow**; set the field **Assigned_Dealer__c = {!Get_Nearest_Dealer.Id}**.

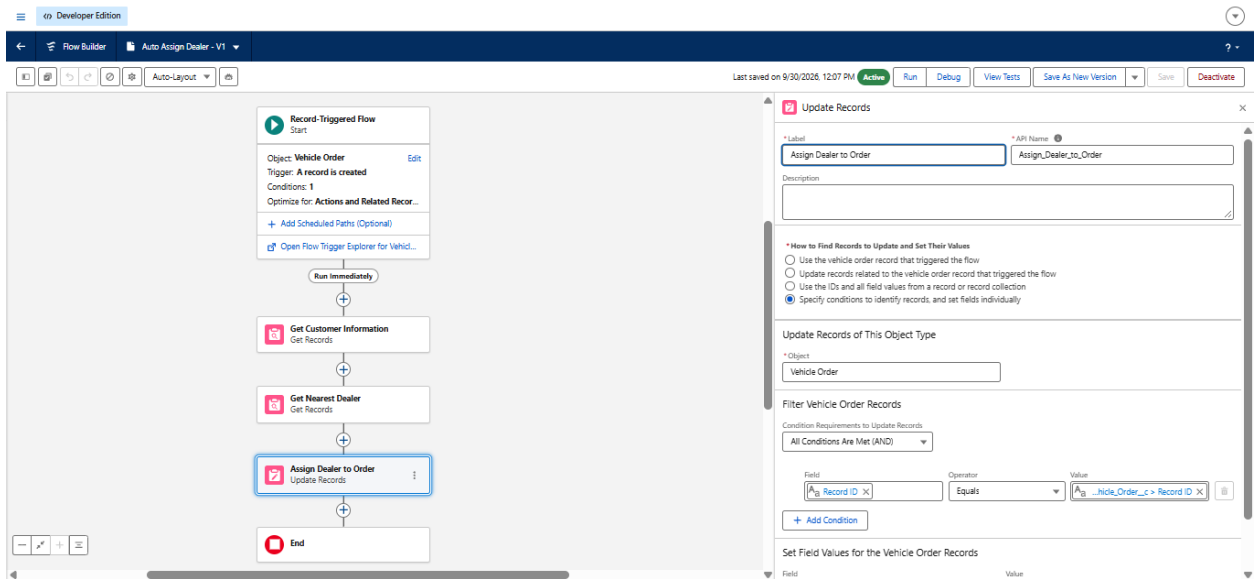


Figure 17: Update Records – Assign Dealer to Order

Step 9: Click **Save**, enter Flow Label = **Auto Assign Dealer** (API name fills in automatically) and click **Save** again.

Step 10: Click **Activate**.

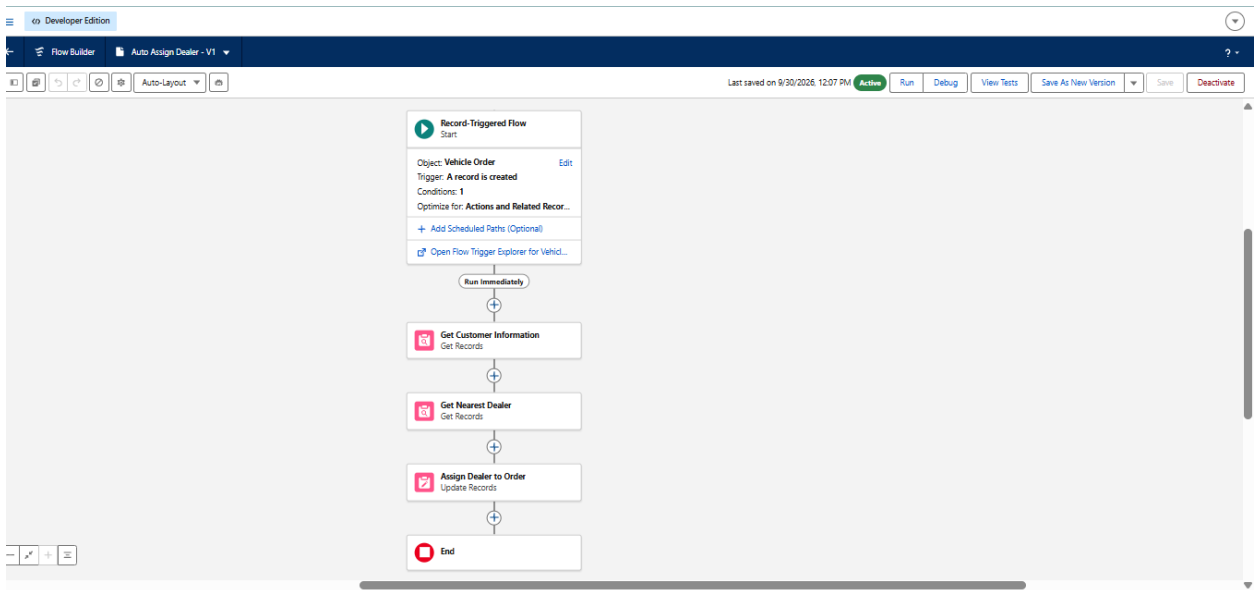


Figure 18: Flow canvas – Auto Assign Dealer (Active)

Flow 2: Test-Drive Reminder Email

This flow e-mails the customer one day before the scheduled test drive.

Step 1: Create a new flow: **New Flow** → **Start From Scratch** → **Record-Triggered Flow** → **Create**.

Step 2: Select Object = **Vehicle Test Drive**; Trigger the flow when **A record is created or updated**; Entry Conditions = **All Conditions Are Met (AND)**: Status__c Equals Scheduled.

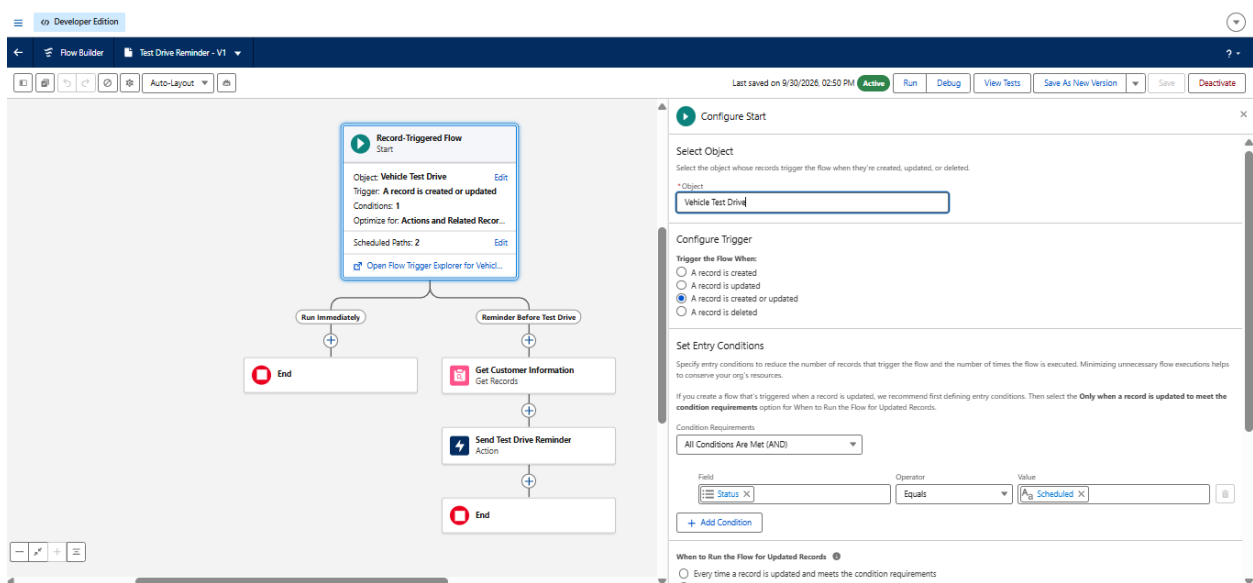


Figure 19: Configure Start – Vehicle Test Drive

Step 3: Click **Add Scheduled Paths (Optional)** below the trigger. Path Label = **Reminder Before Test Drive**; Time Source = **Test_Drive_Date__c**; Offset Number = **1**; Offset Options = **Days Before**. Click **Done**.

Step 4: On the scheduled path click + → **Get Records**. Label = **Get Customer Information**; Object = **Vehicle Customer**; Condition: Id Equals {!\$Record.Customer__c}; **Only the first record**; **Automatically store all fields**.

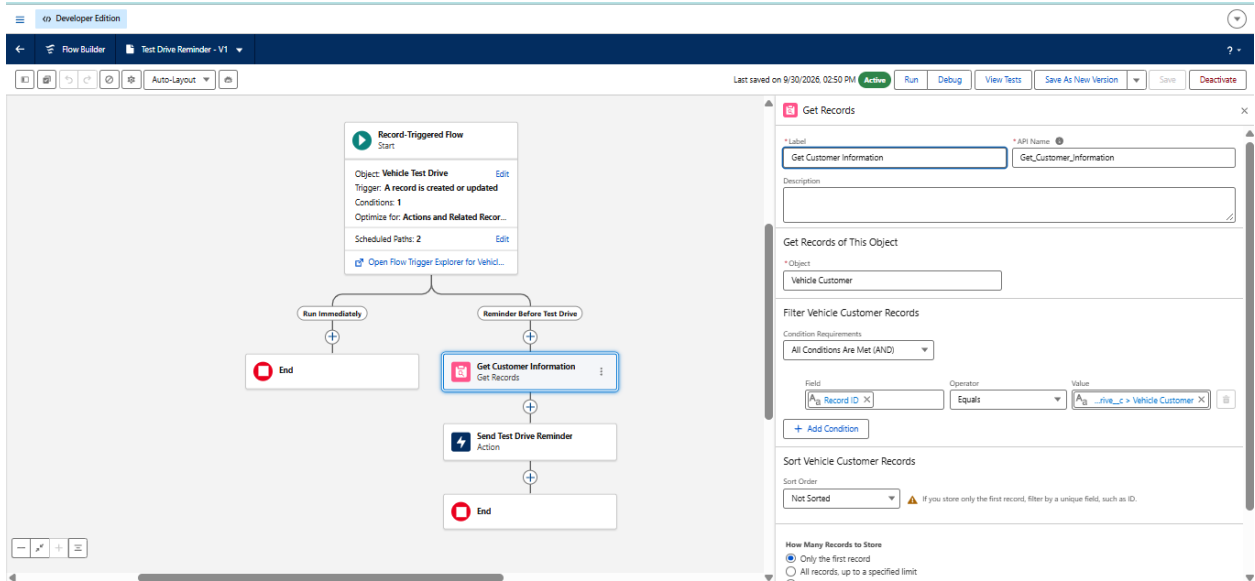


Figure 20: Get Customer Information Configure

Step 5: Add an Action → **Send Email**. Label = **Send Test Drive Reminder**; Subject = "Reminder: Your Test Drive is Tomorrow!"; Recipient Address List = {!Get_Customer_Information.Email__c}; Rich-Text-Formatted Body = **True**.

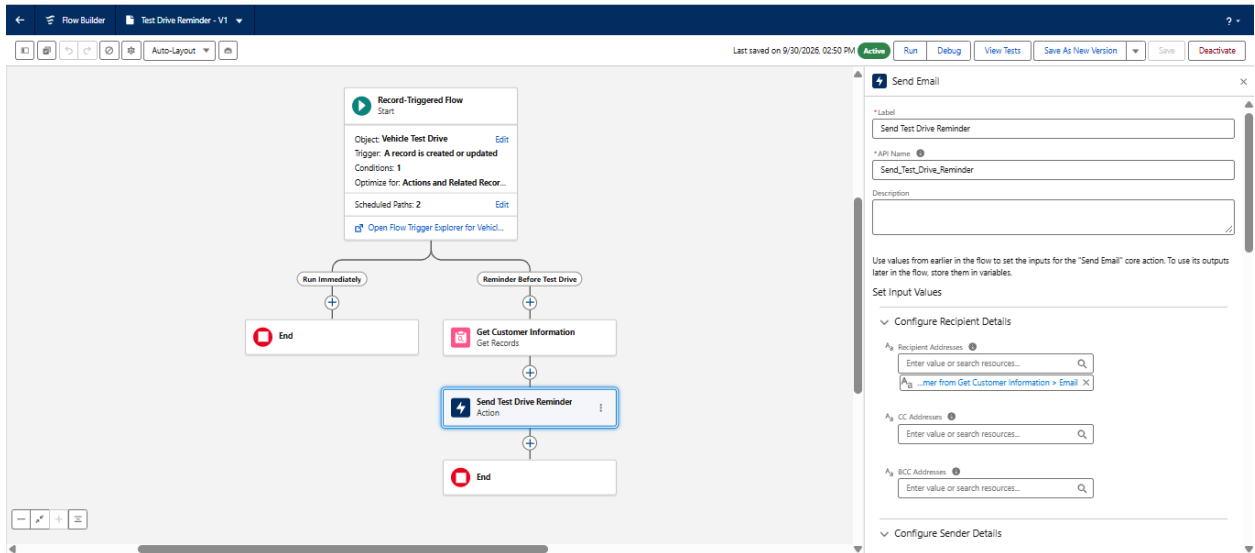


Figure 21: Send Email action configuration

Step 6: For the Body, create a **Text Template** resource named **EmailSent** containing the message, for example: "Dear {!Get_Customer_Information.Name}, this is a reminder that your

test drive is scheduled for tomorrow. If you need to reschedule, please contact us. Thank you, WhatNext Vision Motors Team." Select this resource as the Body.

Step 7: Click **Save**, give the Flow Label **Test Drive Reminder** and click **Save**. Then click **Activate**.

Step 8: Test the flow. Create a test drive with Status = Scheduled and a Test Drive Date of tomorrow (the customer must have a valid email). The reminder is sent at the scheduled time.

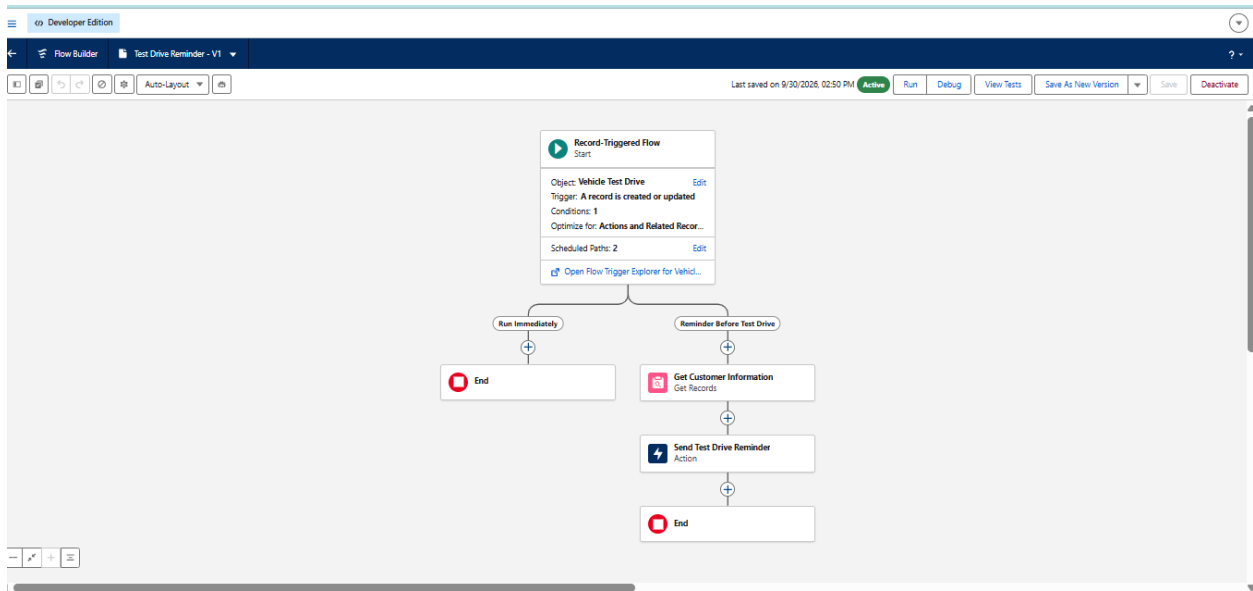


Figure 22: Flow canvas – Test Drive Reminder (Active)

Milestone 7: Apex Trigger, Batch Class and Scheduler

Apex is used where more complex logic is needed than a flow can offer. Three pieces are built: a trigger with a handler class (stock validation and reduction), a batch class (confirming pending orders) and a scheduler (running the batch every night).

Business rules implemented

- A new order cannot be saved when the selected vehicle has no stock.
- When an order becomes **Confirmed**, the stock of that vehicle reduces by one for each confirmed order.
- Orders that remain **Pending** are confirmed by the nightly batch job, in the order they were created, while stock lasts.

Step 1 – Open the Developer Console

Click the gear icon and choose **Developer Console**.

Step 2 – Create the Trigger Handler Class

In the Developer Console click **File** → **New** → **Apex Class**, enter the class name **VehicleOrderTriggerHandler** and click OK.

Replace the default content with the code below and press **Ctrl + S** to save.

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders,
                                     Map<Id, Vehicle_Order__c> oldOrders,
                                     Boolean isBefore, Boolean isAfter,
                                     Boolean isInsert, Boolean isUpdate) {
        if (isBefore && (isInsert || isUpdate)) {
            preventOrderIfOutOfStock(newOrders, oldOrders, isUpdate);
        }
        if (isAfter && (isInsert || isUpdate)) {
            reduceStockOnConfirmation(newOrders, oldOrders, isUpdate);
        }
    }

    // True when the order is newly inserted as Confirmed,
    // or when its status has just changed to Confirmed.
    private static Boolean becameConfirmed(Vehicle_Order__c ord,
                                          Map<Id, Vehicle_Order__c> oldOrders,
                                          Boolean isUpdate) {
        if (ord.Status__c != 'Confirmed') return false;
        if (!isUpdate) return true;
        Vehicle_Order__c oldOrd = oldOrders.get(ord.Id);
        return oldOrd == null || oldOrd.Status__c != 'Confirmed';
    }

    // Block the order when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders,
                                                Map<Id, Vehicle_Order__c> oldOrders,
                                                Boolean isUpdate) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c ord : orders) {
            Boolean needsCheck = isUpdate ? becameConfirmed(ord, oldOrders, true)
                                          : ord.Status__c != 'Canceled';
            if (needsCheck && ord.Vehicle__c != null) {
                vehicleIds.add(ord.Vehicle__c);
            }
        }
    }
}
```

```

    }
}
if (vehicleIds.isEmpty()) return;

Map<Id, Decimal> available = new Map<Id, Decimal>();
for (Vehicle__c v : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
                     WHERE Id IN :vehicleIds]) {
    available.put(v.Id, v.Stock_Quantity__c == null ? 0 : v.Stock_Quantity__c);
}

for (Vehicle_Order__c ord : orders) {
    Boolean needsCheck = isUpdate ? becameConfirmed(ord, oldOrders, true)
                                   : ord.Status__c != 'Canceled';
    if (needsCheck && ord.Vehicle__c != null
        && available.containsKey(ord.Vehicle__c)) {
        if (available.get(ord.Vehicle__c) <= 0) {
            ord.addError('This vehicle is out of stock. Order cannot be placed.');
        } else if (becameConfirmed(ord, oldOrders, isUpdate)) {
            available.put(ord.Vehicle__c, available.get(ord.Vehicle__c) - 1);
        }
    }
}
}

// Reduce vehicle stock by one for every order that has just been confirmed
private static void reduceStockOnConfirmation(List<Vehicle_Order__c> orders,
                                              Map<Id, Vehicle_Order__c> oldOrders,
                                              Boolean isUpdate) {
    Map<Id, Integer> confirmedCount = new Map<Id, Integer>();
    for (Vehicle_Order__c ord : orders) {
        if (ord.Vehicle__c != null && becameConfirmed(ord, oldOrders, isUpdate)) {
            Integer current = confirmedCount.containsKey(ord.Vehicle__c)
                            ? confirmedCount.get(ord.Vehicle__c) : 0;
            confirmedCount.put(ord.Vehicle__c, current + 1);
        }
    }
    if (confirmedCount.isEmpty()) return;

    List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
    for (Vehicle__c v : [SELECT Id, Stock_Quantity__c, Status__c FROM Vehicle__c
                         WHERE Id IN :confirmedCount.keySet()]) {
        Decimal stock = v.Stock_Quantity__c == null ? 0 : v.Stock_Quantity__c;
        stock = stock - confirmedCount.get(v.Id);
        v.Stock_Quantity__c = stock < 0 ? 0 : stock;
        if (v.Stock_Quantity__c == 0) {
            v.Status__c = 'Out of Stock';
        }
        vehiclesToUpdate.add(v);
    }
    update vehiclesToUpdate;
}
}

```

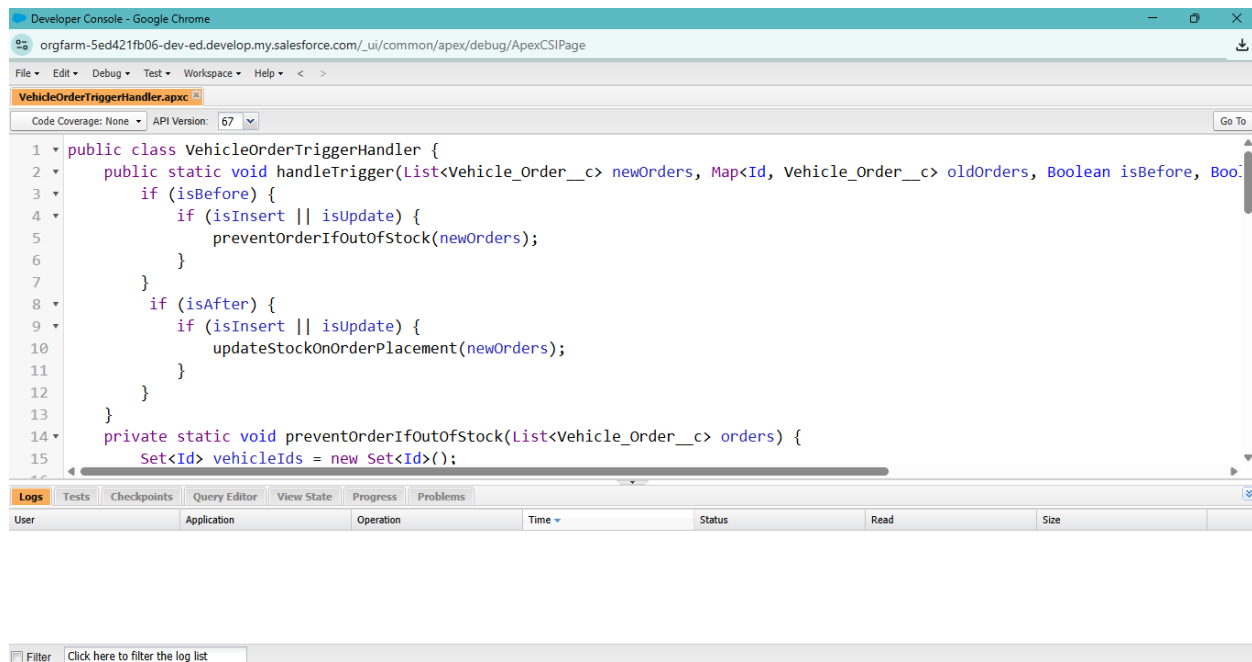


Figure 23: VehicleOrderTriggerHandler in the Developer Console

Step 3 – Create the Apex Trigger

Click **File** → **New** → **Apex Trigger**, enter the name **VehicleOrderTrigger**, choose the sObject **Vehicle_Order__c** and click **Submit**.

Replace the default code with the following, which simply calls the handler:

```

trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update,
                                         after insert, after update) {
    VehicleOrderTriggerHandler.handleTrigger(
        Trigger.new, Trigger.oldMap,
        Trigger.isBefore, Trigger.isAfter,
        Trigger.isInsert, Trigger.isUpdate);
}

```

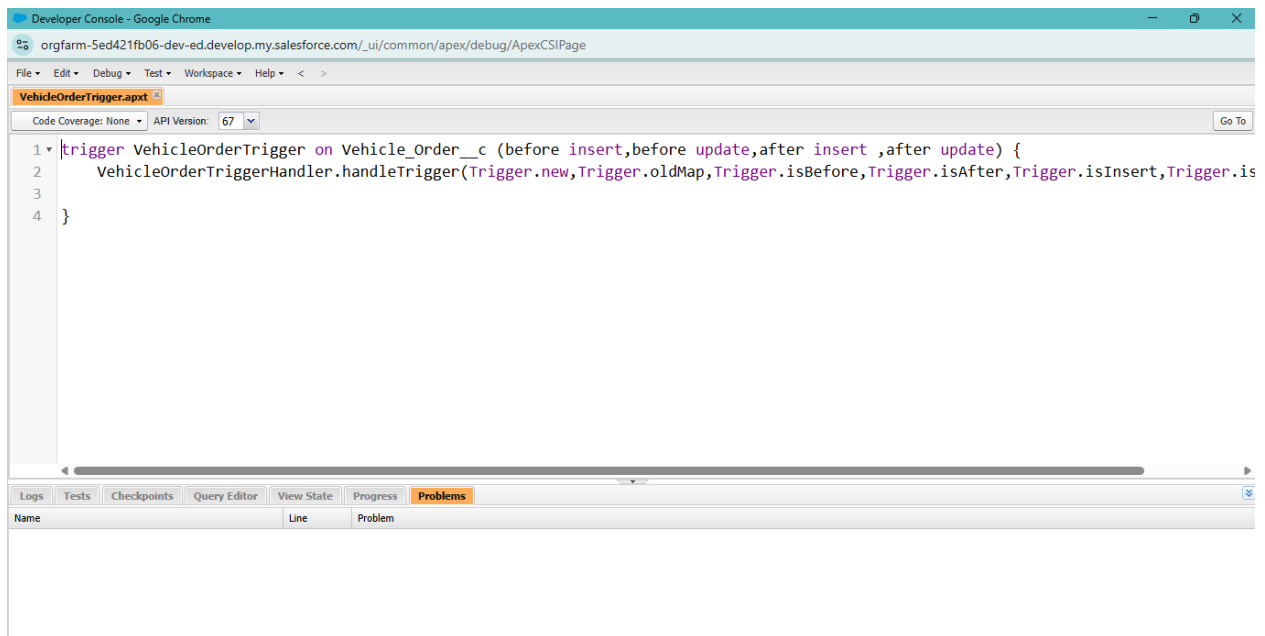


Figure 24: VehicleOrderTrigger code

Step 4 – Test the Stock Validation

Create a Vehicle Order for a vehicle whose Stock Quantity is 0. Salesforce must stop the save and display the message "This vehicle is out of stock. Order cannot be placed."

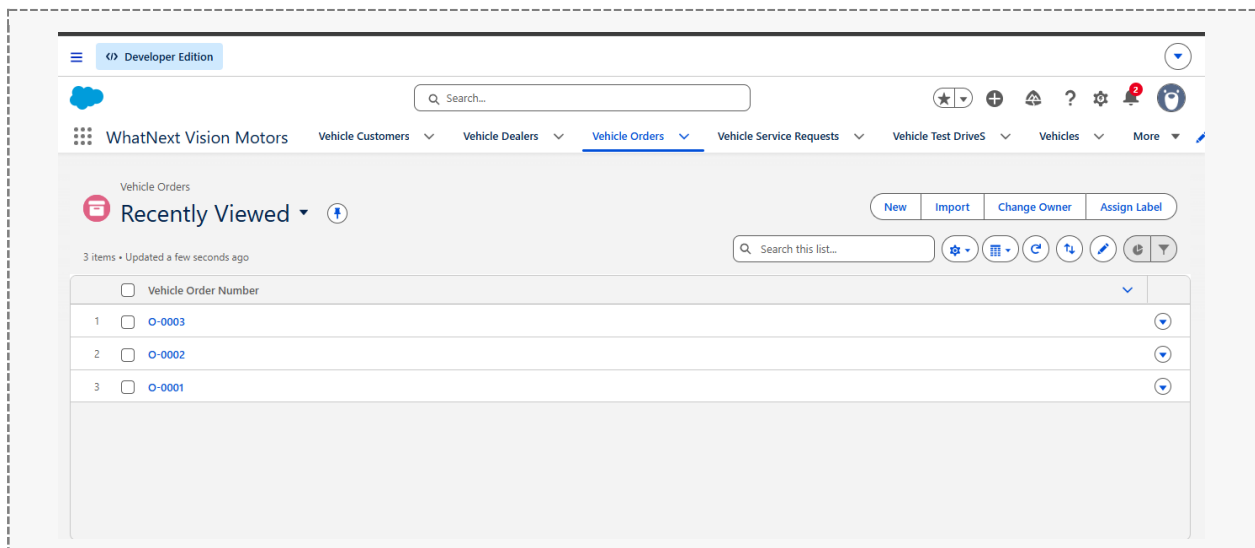


Figure 25: Out-of-stock error message

Next create an order for a vehicle that has stock, set the Status to **Confirmed**, and check that the vehicle's Stock Quantity reduces by one.

Step 5 – Create the Batch Class

Scenario: an order is saved as Pending. When the vehicle has stock, the nightly batch job moves it to Confirmed (the trigger then reduces the stock). If stock is not available the order stays Pending until the vehicle is restocked.

Create an Apex class named **VehicleOrderBatch**.

```
global class VehicleOrderBatch implements Database.Batchable<SObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending' AND Vehicle__c != null
            ORDER BY CreatedDate
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c ord : orderList) {
            vehicleIds.add(ord.Vehicle__c);
        }

        Map<Id, Decimal> available = new Map<Id, Decimal>();
        for (Vehicle__c v : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
            WHERE Id IN :vehicleIds]) {
            available.put(v.Id, v.Stock_Quantity__c == null ? 0 : v.Stock_Quantity__c);
        }

        List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
        for (Vehicle_Order__c ord : orderList) {
            Decimal stock = available.get(ord.Vehicle__c);
            if (stock != null && stock > 0) {
                ord.Status__c = 'Confirmed';
                available.put(ord.Vehicle__c, stock - 1);
                ordersToUpdate.add(ord);
            }
        }

        // The Apex trigger reduces the vehicle stock when each order is confirmed
        if (!ordersToUpdate.isEmpty()) {
            update ordersToUpdate;
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
```

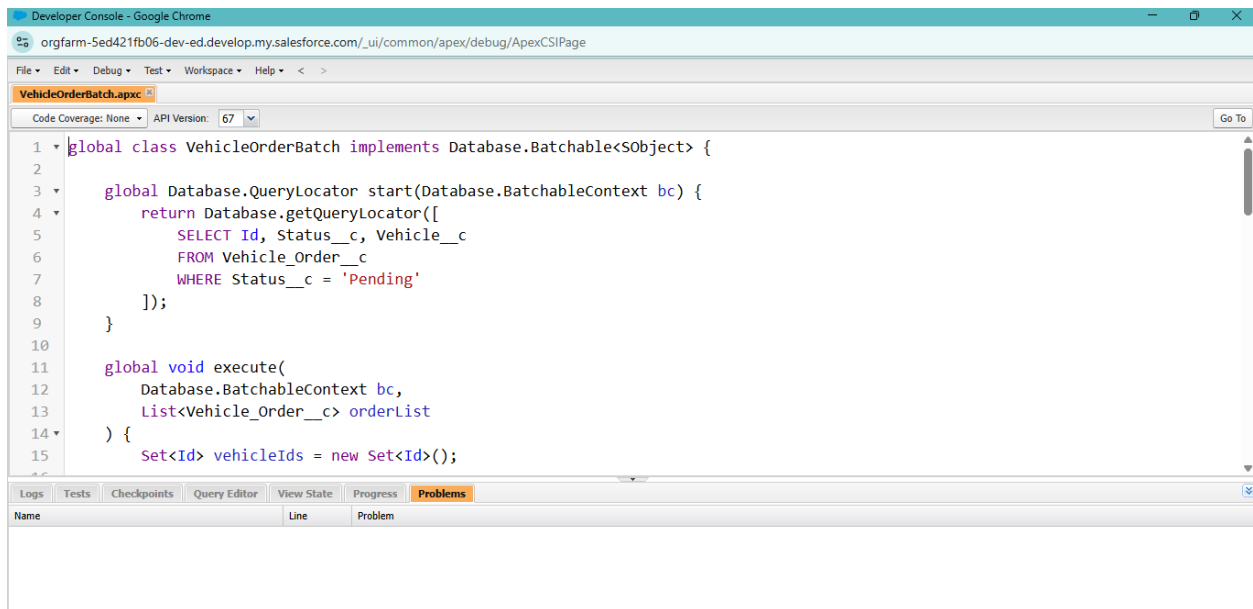


Figure 26: VehicleOrderBatch in the Developer Console

Step 6 – Create the Scheduler Class

Create a class named **VehicleOrderBatchScheduler** that starts the batch job.

```

global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}

```

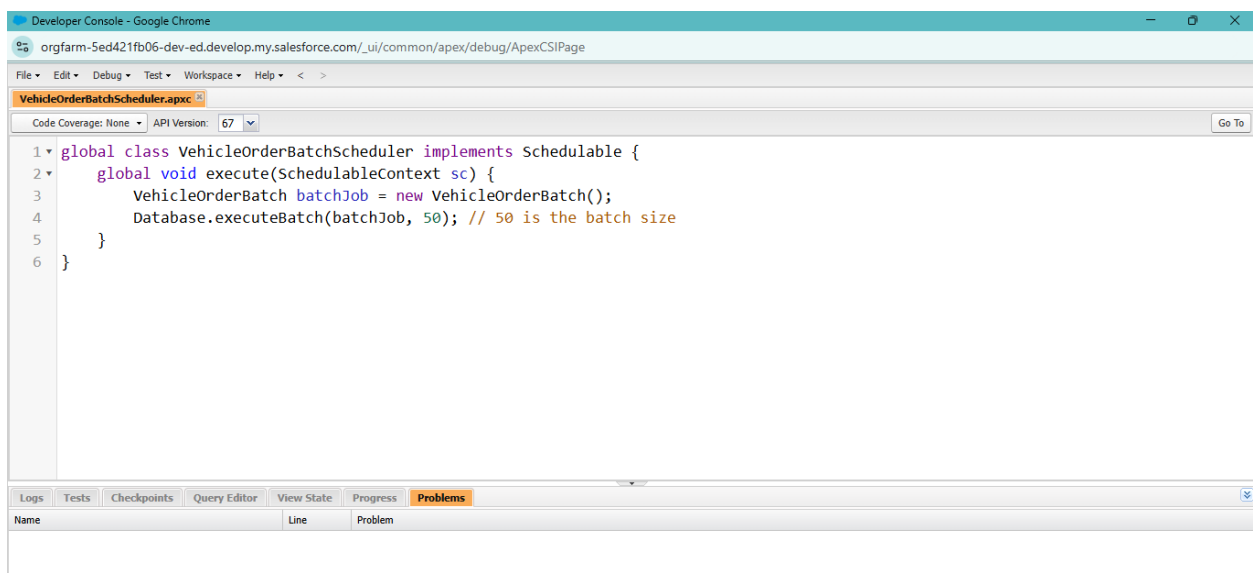


Figure 27: VehicleOrderBatchScheduler in the Developer Console

Step 7 – Schedule the Batch Job

To run the batch job **every night at midnight**, open **Debug → Open Execute Anonymous Window**, paste the code below and click **Execute**.

```
String cronExp = '0 0 0 * * ?'; // every day at 12:00 AM (midnight)
System.schedule('Nightly Vehicle Order Processing', cronExp, new
VehicleOrderBatchScheduler());
```

Step 8 – Verify the Scheduled Job

In Setup search **Scheduled Jobs** and open it. The job "Nightly Vehicle Order Processing" must appear in the list with its next scheduled run.

To test the batch immediately, run this in the Execute Anonymous window and then check the order statuses:

```
Database.executeBatch(new VehicleOrderBatch(), 50);
```

Milestone 8: Reports & Dashboards

Reports and dashboards fulfil requirement FR-7 and give managers a live view of orders, dealer performance and sales.

Activity 1: Create a Custom Report Type (optional)

If you want to report on orders together with their vehicles and dealers, create a custom report type: **Setup → Report Types → New Custom Report Type**, primary object **Vehicle Orders**, category **Other Reports**, and deploy it.

Activity 2: Create the Reports

Report	Report type	Grouping / filter	Chart
Orders by Status	Vehicle Orders	Group rows by Status__c	Donut chart
Dealer Performance	Vehicle Orders with Dealer	Group rows by Assigned Dealer; count of orders	Horizontal bar chart
Sales by Vehicle Model	Vehicle Orders with Vehicle	Group by Vehicle Model; filter Status = Confirmed or Delivered	Vertical bar chart
Low Stock Vehicles	Vehicles	Filter Stock Quantity < 5	None (table)

Go to the **Reports** tab → **New Report**, choose the report type, set the grouping and filters above, then **Save & Run**.

Activity 3: Create the Dashboard

Step 1: Open the **Dashboards** tab → **New Dashboard**, name it **WhatNext Vision Motors – Sales Overview** and click **Create**.

Step 2: Click + **Component**, choose a report from the list above, pick a chart style and click **Add**. Repeat for all four reports.

Step 3: Click **Save** and then **Done**.

7. FUNCTIONAL & PERFORMANCE TESTING

During testing, screenshots were captured from the Salesforce org to validate vehicle creation, dealer assignment, stock-validation triggers, order status updates, batch processing, scheduled reminders and dashboard analytics. Every feature was checked to confirm correct behaviour and acceptable performance.

7.1 Functional Test Cases

ID	Test scenario	Test steps	Expected result	Result
TC-01	Create a vehicle	Open Vehicles tab → New → enter model, price, stock → Save	Record saved with Status Available	Pass
TC-02	Create a dealer	Open Vehicle Dealers tab → New → enter name, location → Save	Record saved; Dealer Code auto-generated	Pass
TC-03	Create a customer	Open Vehicle Customers tab → New → enter name, email, address → Save	Record saved	Pass
TC-04	Order with stock available	Create a Vehicle Order (Pending) for a vehicle with stock	Order saved; Assigned Dealer filled by flow	Pass
TC-05	Order for out-of-stock vehicle	Create an order for a vehicle with stock = 0	Error: "This vehicle is out of stock"	Pass
TC-06	Stock reduction	Change an order to Confirmed	Vehicle stock reduces by 1	Pass
TC-07	Batch confirmation	Run VehicleOrderBatch with Pending orders and stock	Orders become Confirmed	Pass
TC-08	Scheduled job	Open Setup → Scheduled Jobs	Nightly job is listed	Pass
TC-09	Test-drive reminder	Create a Scheduled test drive for tomorrow	Reminder email received	Pass
TC-10	Reports & dashboard	Open the dashboard	Charts show current data	Pass

Update the Result column with the outcome you actually observed in your org.

7.2 Test Evidence (Screenshots)

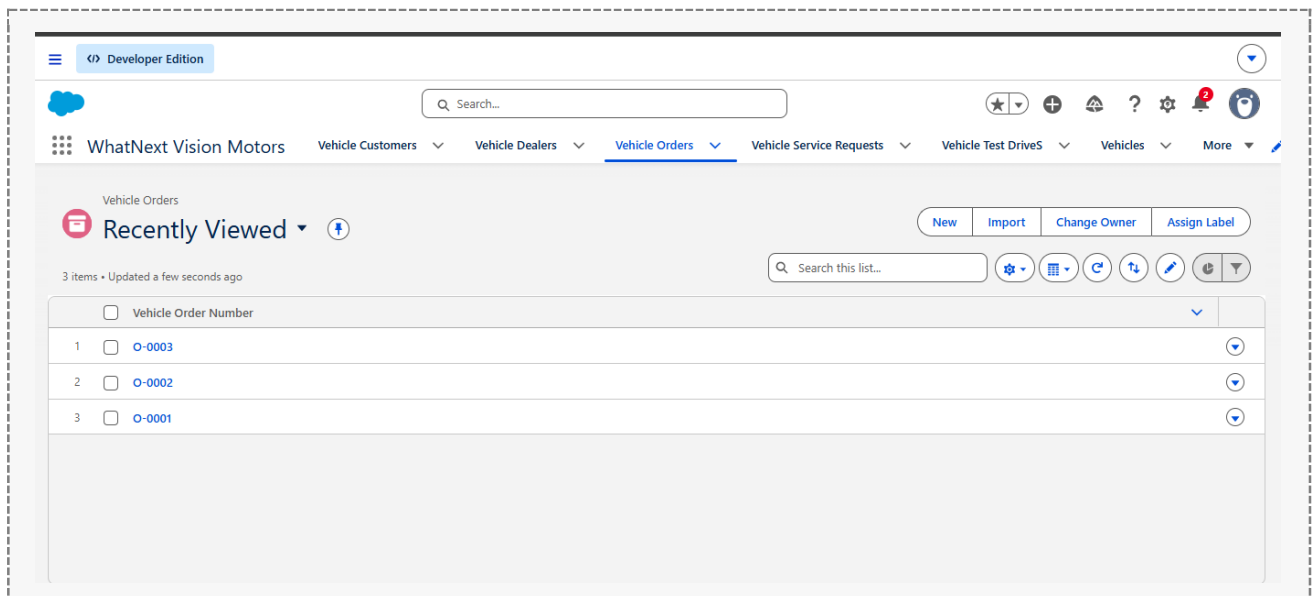


Figure 28: TC-01 to TC-03 – Vehicle, dealer and customer records created

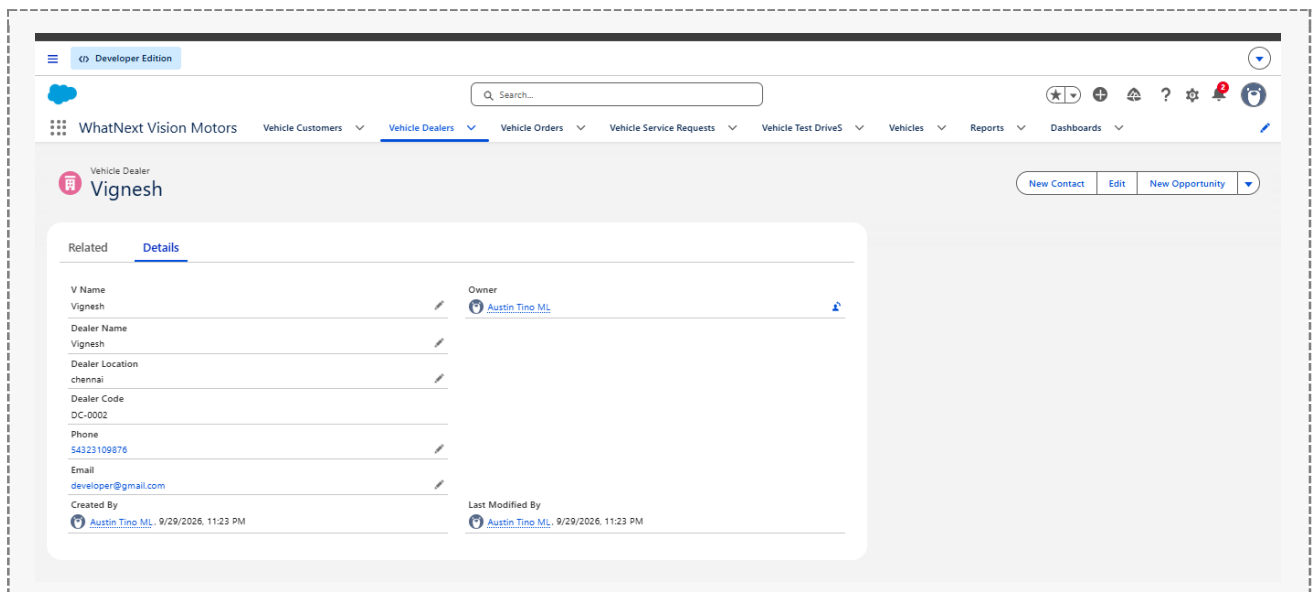


Figure 29: TC-04 – Dealer auto-assigned to order

7.3 Performance Testing

Performance was checked by measuring how the automation behaves with a larger number of records. Record the values you observe in your org in the "Observed" column.

Test	Load	Target	Observed
Create a single order (trigger + flow)	1 record	Saves in under 3 seconds	[Enter value]
Bulk order insert through Data Loader	200 records	No governor-limit error	[Enter value]

Test	Load	Target	Observed
Batch job execution	All Pending orders, batch size 50	Completes without failures	[Enter value]
Dashboard refresh	4 components	Loads in under 5 seconds	[Enter value]

8. ADVANTAGES & DISADVANTAGES

Advantages

- **Centralized data:** vehicles, dealers, customers and orders are in one place.
- **Automated workflows:** dealer assignment, stock checks and reminders need no manual effort.
- **Real-time reporting:** dashboards show current order and stock information.
- **Fewer errors:** out-of-stock orders are blocked before they are saved.
- **Scalable:** more dealers, vehicles and locations can be added without redesign.

Disadvantages

- **Platform dependency:** the solution runs only on Salesforce and needs licences for production use.
- **Initial configuration effort:** objects, fields, flows and Apex must be set up and tested.
- **Simple dealer matching:** dealers are matched by location text, not by real distance.
- **Governor limits:** very large data volumes need careful batch sizing.

9. CONCLUSION

The Salesforce CRM implementation for WhatNext Vision Motors modernizes vehicle ordering and dealership management. By combining custom objects, record-triggered flows, Apex triggers and scheduled batch processing, the project replaces manual registers with a single automated platform. The result is higher efficiency, more accurate orders and a better experience for customers, dealers and managers.

10. FUTURE SCOPE

- Mobile ordering application for customers
- GPS-based dealer routing using real distance instead of location text
- SMS and WhatsApp notifications
- AI-based demand forecasting for stock planning
- Online payment and finance integration
- Customer portal using Experience Cloud

11. APPENDIX

- **Source code:** Salesforce configuration and Apex code (listed below and in Milestone 7)
- **Dataset:** Salesforce custom objects and the sample records created in the org
- **Demo / project link:** Salesforce Developer Org – [paste your org URL or demo video link here]

A. Apex Test Class (for code coverage)

Salesforce requires at least 75% code coverage before Apex can be deployed to production. The following test class covers the handler, the trigger and the batch class.

```
@isTest
private class VehicleOrderTests {

    @testSetup
    static void setup() {
        Vehicle_Customer__c cust = new Vehicle_Customer__c(Name = 'Test Customer');
        insert cust;
        insert new Vehicle__c(Name = 'Test SUV', Stock_Quantity__c = 1,
                               Status__c = 'Available');
        insert new Vehicle__c(Name = 'Empty Sedan', Stock_Quantity__c = 0,
                               Status__c = 'Out of Stock');
    }

    @isTest
    static void blocksOrderWhenOutOfStock() {
        Vehicle_Customer__c cust = [SELECT Id FROM Vehicle_Customer__c LIMIT 1];
        Vehicle__c veh = [SELECT Id FROM Vehicle__c WHERE Name = 'Empty Sedan'];
        Vehicle_Order__c ord = new Vehicle_Order__c(
            Customer__c = cust.Id, Vehicle__c = veh.Id,
            Order_Date__c = Date.today(), Status__c = 'Pending');
        Database.SaveResult res = Database.insert(ord, false);
        System.assert(!res.isSuccess(), 'Order should be blocked');
    }

    @isTest
    static void batchConfirmsPendingOrderAndReducesStock() {
        Vehicle_Customer__c cust = [SELECT Id FROM Vehicle_Customer__c LIMIT 1];
        Vehicle__c veh = [SELECT Id FROM Vehicle__c WHERE Name = 'Test SUV'];
        insert new Vehicle_Order__c(Customer__c = cust.Id, Vehicle__c = veh.Id,
                                     Order_Date__c = Date.today(), Status__c = 'Pending');

        Test.startTest();
        Database.executeBatch(new VehicleOrderBatch(), 50);
        Test.stopTest();

        Vehicle_Order__c ord = [SELECT Status__c FROM Vehicle_Order__c LIMIT 1];
        System.assertEquals('Confirmed', ord.Status__c);
        Vehicle__c updated = [SELECT Stock_Quantity__c FROM Vehicle__c WHERE Id = :veh.Id];
        System.assertEquals(0, updated.Stock_Quantity__c);
    }
}
```

B. Object & Field Summary

Object	Key fields
Vehicle__c	Vehicle_Model__c, Stock_Quantity__c, Price__c, Dealer__c, Status__c

Object	Key fields
Vehicle_Dealer__c	Dealer_Location__c, Dealer_Code__c, Phone__c, Email__c
Vehicle_Customer__c	Email__c, Phone__c, Address__c, Preferred_Vehicle_Type__c
Vehicle_Order__c	Customer__c, Vehicle__c, Assigned_Dealer__c, Order_Date__c, Status__c
Vehicle_Test_Drive__c	Customer__c, Vehicle__c, Test_Drive_Date__c, Status__c
Vehicle_Service_Request__c	Customer__c, Vehicle__c, Service_Date__c, Issue_Description__c, Status__c

C. Automation Summary

Component	Type	Purpose
Auto Assign Dealer	Record-Triggered Flow	Fills Assigned Dealer when an order is created
Test Drive Reminder	Record-Triggered Flow (scheduled path)	Emails the customer one day before the test drive
VehicleOrderTrigger / Handler	Apex Trigger + Class	Blocks out-of-stock orders; reduces stock on confirmation
VehicleOrderBatch	Batch Apex	Confirms Pending orders when stock is available
VehicleOrderBatchScheduler	Scheduled Apex	Runs the batch every night at midnight

D. References

- Salesforce Developers – <https://developer.salesforce.com>
- Salesforce Help – Flow Builder and Record-Triggered Flows
- Apex Developer Guide – Triggers, Batch Apex and Scheduled Apex
- Trailhead – Apex Basics & Database, Flow Basics, Data Modeling